Laporan UTS
3D Mapping Menggunakan Drone pada CoppeliaSim

Ademas Fazri Sunaryo
NRP 5022241066
Departemen Teknik Elektro, Institut Teknologi Sepuluh Nopember (ITS), Surabaya
Oktober 2026

---

1 Pendahuluan

Laporan ini menjelaskan sistem *3D mapping* berbasis drone quadcopter yang disimulasikan di CoppeliaSim dan dikendalikan melalui Python (ZMQ Remote API). Drone terbang menyapu area dengan pola *lawnmower*, memindai lingkungan dengan sensor 3D laser (*fast3DLaserScanner*), mentransformasikan titik hasil scan dari frame sensor ke frame global, lalu memproses awan titik (*point cloud*) menjadi peta 3D yang diberi warna berdasarkan ketinggian.

Tujuan pekerjaan:

1. Mengontrol drone sepanjang jalur terbang dan menampilkan jalur yang dilalui.
2. Melakukan scan 3D dengan sensor yang dipasang pada drone, dengan posisi dan orientasi yang memungkinkan sensor menangkap lantai, badan objek, dan puncak objek.
3. Mereduksi jumlah titik dengan *voxel downsampling* sebagai post-process.
4. Memberi *color mapping* ketinggian pada peta sebagai post-process.
5. Menambah kompleksitas berupa perencanaan ketinggian yang sadar rintangan dan pembatas jarak target.

2 Setup Simulasi

Lingkungan. Arena berukuran $4\,\text{m} \times 4\,\text{m}$ dikelilingi *ExternalWall*. Di dalamnya terdapat dua rintangan: *sofa* (tinggi puncak $\approx 1{,}0$ m) dan *ConcreteBlock* dengan manekin di atasnya (tinggi puncak $\approx 2{,}0$ m). Variasi tinggi ini sengaja dipilih agar sensor harus menangkap tiga hal sekaligus: ground, badan objek, dan puncak objek.

Robot. Model *Quadcopter* bawaan CoppeliaSim dengan tiga objek penting: `base` (frame badan drone), `target` (titik acuan yang diikuti drone), dan `fast3DLaserScanner/sensor` (sensor yang terpasang rigid pada drone).

3 Arsitektur Sistem

Simulasi dijalankan dalam mode *stepping* sinkron, sehingga satu iterasi program tepat sama dengan satu *time step* simulasi Δt. Pose drone dan data sensor dengan demikian selalu berasal dari waktu yang sama. Alur kerja keseluruhan ditunjukkan pada Gambar 2.

Tabel 1: Parameter utama sistem.

Parameter	Simbol	Nilai
Area scan	X, Y	$[-2,2] \times [-2,2]$ m
Ketinggian terbang dasar	z_0	1,5 m
Jarak antar jalur	g	1,0 m
Kecepatan target	v	0,4 m/s
Toleransi waypoint	ε	0,15 m
Jangkauan sensor	$R_{\max}$	5,0 m
Ukuran voxel	s_v	0,05 m
Margin keamanan horizontal	m	0,6 m
Clearance vertikal	h_c	0,6 m
Panjang ramp	L_r	1,0 m
Jarak maksimum target–drone	d_{lead}	0,5 m

4 Perencanaan Jalur

4.1 Pola Lawnmower

Waypoint disusun sebagai garis-garis sejajar sumbu Y yang dilalui bolak-balik. Untuk jalur ke-i:

$$x_i = x_{\min} + i\,g, \qquad i = 0, 1, \ldots, \left\lfloor \frac{x_{\max} - x_{\min}}{g} \right\rfloor \tag{1}$$

Arah y dibalik pada setiap jalur (zig-zag):

$$\text{jalur } i: \quad \begin{cases} (x_i, y_{\min}) \to (x_i, y_{\max}) & i \text{ genap} \\ (x_i, y_{\max}) \to (x_i, y_{\min}) & i \text{ ganjil} \end{cases} \tag{2}$$

Semua waypoint berada pada ketinggian z_0. Satu waypoint awal ditambahkan di atas titik start agar drone naik terlebih dahulu sebelum bergerak lateral.

4.2 Interpolasi Padat

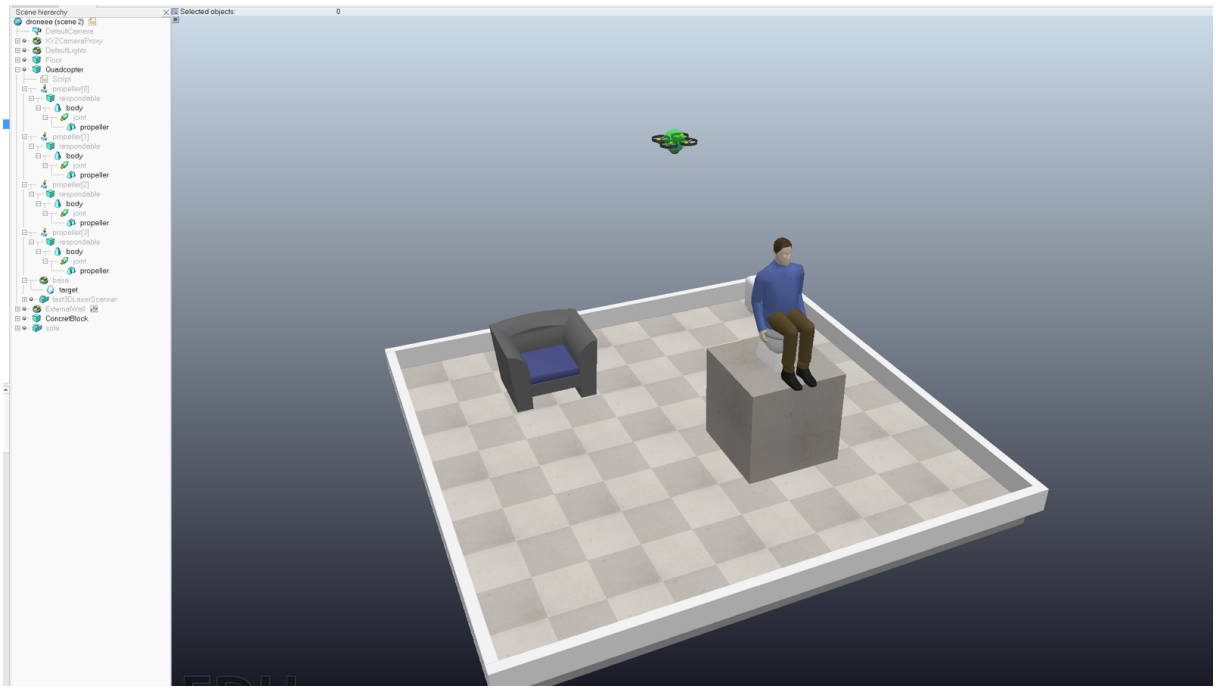
Setiap segmen dari $\boldsymbol{a}$ ke $\boldsymbol{b}$ dipecah menjadi n titik berjarak seragam ($\Delta s = 0{,}2$ m):

$$n = \max\left(\left\lfloor \frac{\|\boldsymbol{b} - \boldsymbol{a}\|}{\Delta s} \right\rfloor, 1\right), \qquad \boldsymbol{p}_k = \boldsymbol{a} + (\boldsymbol{b} - \boldsymbol{a})\frac{k}{n}, \quad k = 1, \ldots, n \tag{3}$$

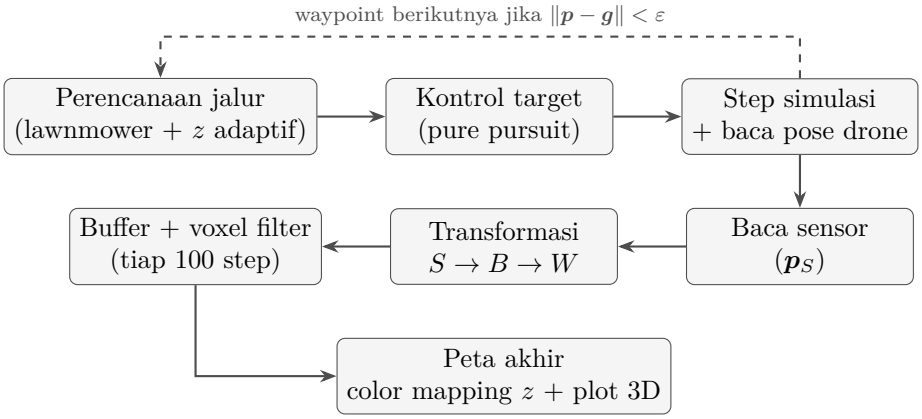
4.3 Ketinggian Adaptif terhadap Rintangan

Jika jalur dibiarkan pada ketinggian tetap, drone dapat menabrak objek yang lebih tinggi dari z_0 (misalnya ConcreteBlock 2,0 m). Karena itu setiap titik $\boldsymbol{p}_i$ diperiksa terhadap kotak 2D tiap rintangan yang diperlebar margin m. Jika berada di dalamnya, ketinggian minimum dinaikkan:

$$z_i^{\text{need}} = \begin{cases} \max(z_i,\ h_{\text{obs}} + h_c) & x_0 - m \le x_i \le x_1 + m \ \wedge\ y_0 - m \le y_i \le y_1 + m \\ z_i & \text{selain itu} \end{cases} \tag{4}$$



Gambar 1: Scene CoppeliaSim: drone Quadcopter, arena 4×4 m, sofa, ConcreteBlock dengan manekin, dan scene hierarchy.



Gambar 2: Diagram alur sistem 3D mapping.

Sebagai contoh, di atas sofa $z^{\text{need}} = 1{,}0 + 0{,}6 = 1{,}6$ m, sedangkan di atas ConcreteBlock $z^{\text{need}} = 2{,}0 + 0{,}6 = 2{,}6$ m.

Kenaikan mendadak tidak dapat diikuti drone, sehingga dibuat *ramp* dengan operasi *sliding-window maximum* (dilasi 1D) selebar $k = \lceil L_r/\Delta s \rceil = 5$ titik di kiri dan kanan:

$$z_i = \max_{i-k \le j \le i+k} z_j^{\text{need}} \tag{5}$$

Hasilnya, drone mulai naik sekitar 1 m sebelum mencapai rintangan dan baru turun sekitar 1 m setelah melewatinya. Pola ini terlihat pada jalur biru di peta 3D (Gambar 4).

5 Kontrol Drone

Drone tidak dikendalikan secara langsung. Drone mengikuti objek `target` melalui kontroler bawaan model quadcopter, sehingga program hanya perlu menggerakkan `target` sepanjang jalur. Pada tiap time step, posisi target $\boldsymbol{t}$ diperbarui dengan kecepatan konstan v menuju waypoint aktif $\boldsymbol{g}$:

$$d = \|\boldsymbol{g} - \boldsymbol{t}\|, \qquad \boldsymbol{t} \leftarrow \begin{cases} \boldsymbol{g} & d < v\,\Delta t \\ \boldsymbol{t} + \dfrac{\boldsymbol{g} - \boldsymbol{t}}{d}\, v\,\Delta t & \text{selain itu} \end{cases} \tag{6}$$

Pembatas jarak (*lead limiter*). Target hanya boleh maju jika drone masih cukup dekat:

$$\|\boldsymbol{t} - \boldsymbol{p}_{\text{drone}}\| < d_{\text{lead}} = 0{,}5 \text{ m} \tag{7}$$

Tanpa pembatas ini, target dapat berlari jauh meninggalkan drone, terutama saat drone harus naik di atas rintangan, sehingga drone memotong jalur atau tertinggal jauh. Dengan pembatas, target "menunggu" drone dan pelacakan jalur menjadi lebih akurat.

Kriteria waypoint tercapai:

$$\|\boldsymbol{p}_{\text{drone}} - \boldsymbol{g}\| < \varepsilon = 0{,}15 \text{ m} \implies \text{indeks waypoint bertambah} \tag{8}$$

Loop berhenti saat seluruh waypoint telah dilalui. Posisi drone dicatat tiap step sebagai jalur terbang, yang kemudian digambar sebagai garis biru pada peta.

6 Sensor 3D dan Pembacaan Data

Sensor yang digunakan adalah *fast3DLaserScanner*, sensor berbasis kamera kedalaman yang mengembalikan koordinat 3D setiap piksel pada frame sensor. Sensor dipasang rigid pada drone dengan posisi dan orientasi yang diatur agar daerah pandangnya mencakup lantai di bawah drone sekaligus sisi dan puncak objek yang dilewati. Hal ini terlihat pada Gambar 3: titik merah menempel pada lantai, dinding samping ConcreteBlock, dan permukaan atasnya.

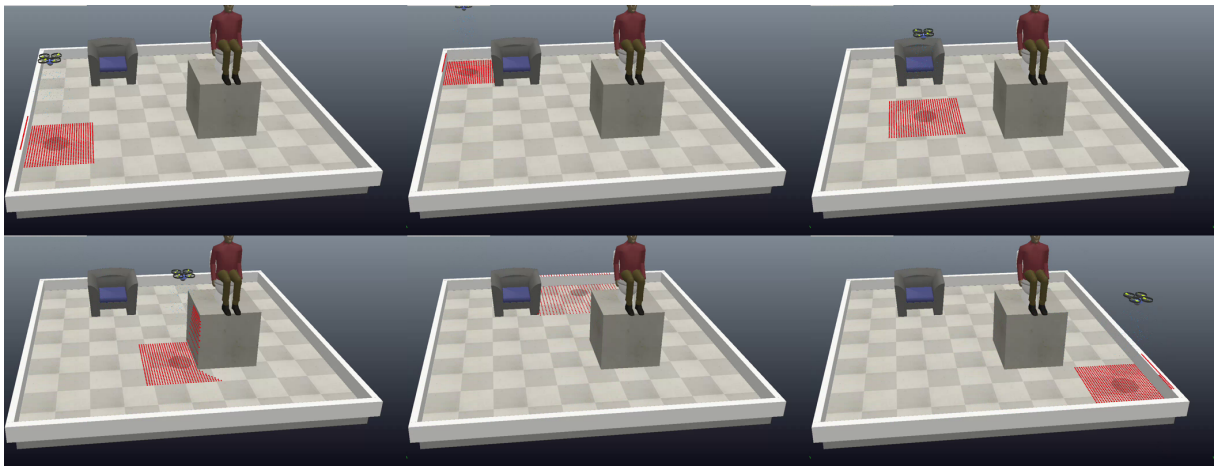
Setiap *frame* sensor dikembalikan dalam bentuk paket: lebar w, tinggi h, lalu $w \cdot h$ elemen berisi empat nilai (x, y, z, d), dengan d jarak terukur. Data diparsing menjadi larik $N \times 4$ dan hanya titik valid yang dipakai:

$$\text{valid} \iff d < 0{,}9999\, R_{\max} \tag{9}$$

Piksel yang tidak mengenai apa pun mengembalikan $d = R_{\max}$, sehingga harus dibuang agar tidak muncul titik palsu di tepi jangkauan sensor. Tiga step pertama simulasi dilewati karena buffer sensor belum berisi data valid.

7 Transformasi Koordinat: Sensor → Base → World

Titik hasil pembacaan berada pada frame sensor $\{S\}$, sedangkan peta harus berada pada satu frame global $\{W\}$. Transformasi dilakukan dalam dua tahap melalui frame badan drone $\{B\}$.



Gambar 3: Urutan scan drone di arena (kiri ke kanan, atas ke bawah). Titik merah adalah hasil scan sensor pada tiap posisi drone.

7.1 Matriks Transformasi Homogen

Pose suatu frame relatif terhadap frame lain dinyatakan dengan matriks homogen 4×4:

$$\boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\top} & 1 \end{bmatrix}, \qquad \boldsymbol{R} \in SO(3), \;\; \boldsymbol{t} \in \mathbb{R}^3 \tag{10}$$

CoppeliaSim mengembalikan matriks 3×4 $[\boldsymbol{R} \,|\, \boldsymbol{t}]$ dalam urutan baris; program menambahkan baris [0 0 0 1] untuk membentuk $\boldsymbol{T}$. Untuk titik homogen $\tilde{\boldsymbol{p}} = [x \; y \; z \; 1]^{\top}$ berlaku

$$\tilde{\boldsymbol{p}}_B = \boldsymbol{T}_B^S \, \tilde{\boldsymbol{p}}_S \qquad \text{(sensor} \rightarrow \text{base)} \tag{11}$$

$$\tilde{\boldsymbol{p}}_W = \boldsymbol{T}_W^B \, \tilde{\boldsymbol{p}}_B \qquad \text{(base} \rightarrow \text{world)} \tag{12}$$

dan gabungan keduanya:

$$\boxed{\tilde{\boldsymbol{p}}_W = \boldsymbol{T}_W^B \, \boldsymbol{T}_B^S \, \tilde{\boldsymbol{p}}_S} \tag{13}$$

Bila diuraikan menjadi rotasi dan translasi:

$$\boldsymbol{p}_W = \boldsymbol{R}_{WB} \left(\boldsymbol{R}_{BS} \, \boldsymbol{p}_S + \boldsymbol{t}_{BS} \right) + \boldsymbol{t}_{WB} \tag{14}$$

7.2 Peran Tiap Matriks

- $\boldsymbol{T}_B^S$ (pose sensor terhadap base) bersifat konstan karena sensor terpasang rigid pada drone. Matriks ini menyatakan posisi dan orientasi sensor yang diatur agar dapat melihat ground, badan objek, dan puncak objek.
- $\boldsymbol{T}_W^B$ (pose base terhadap world) berubah tiap step karena drone bergerak dan berotasi. Matriks ini diambil ulang setiap kali sensor dibaca. Rotasinya (roll, pitch, yaw) memastikan titik tetap jatuh pada lokasi dunia yang benar walaupun drone miring saat berakselerasi.

7.3 Bentuk Vektor untuk Banyak Titik

Untuk N titik yang disimpan sebagai matriks $\boldsymbol{P} \in \mathbb{R}^{N\times 3}$ (satu titik per baris), transformasi dihitung sekaligus tanpa loop:

$$\boldsymbol{P}_W = \boldsymbol{P}\,\boldsymbol{R}^\top + \mathbf{1}\,\boldsymbol{t}^\top \tag{15}$$

dengan $\mathbf{1} \in \mathbb{R}^N$ vektor satuan. Operasi ini dilakukan dua kali (sensor $\rightarrow$ base, lalu base $\rightarrow$ world) pada setiap frame sensor.

8 Post-Processing

8.1 Akumulasi Awan Titik

Titik dari setiap frame ditampung di buffer. Setiap 100 step, buffer digabung ke peta global lalu langsung difilter voxel agar memori tidak membengkak:

$$\mathcal{C}_{k+1} = \mathcal{V}(\mathcal{C}_k \cup \mathcal{B}_k) \tag{16}$$

dengan $\mathcal{C}$ awan titik global, $\mathcal{B}_k$ isi buffer, dan $\mathcal{V}$ operator voxel filter.

8.2 Voxel Downsampling

Ruang 3D dibagi menjadi kubus kecil (voxel) bersisi $s_v = 0{,}05$ m. Setiap titik dipetakan ke indeks voxel:

$$\text{key}(\boldsymbol{p}) = \left\lfloor \frac{\boldsymbol{p}}{s_v} \right\rfloor = \left(\left\lfloor \tfrac{x}{s_v} \right\rfloor, \left\lfloor \tfrac{y}{s_v} \right\rfloor, \left\lfloor \tfrac{z}{s_v} \right\rfloor \right) \in \mathbb{Z}^3 \tag{17}$$

Dari semua titik dengan key yang sama, hanya satu yang dipertahankan (melalui `np.unique` pada key, kompleksitas $O(N \log N)$). Karena sensor melihat area yang sama berulang kali dari banyak posisi, filter ini menghapus duplikasi tanpa mengubah bentuk peta. Beberapa sifatnya:

- Batas atas jumlah titik pada lantai adalah $(4/0{,}05)^2 = 80 \times 80 = 6.400$ titik. Titik tambahan berasal dari permukaan vertikal dan puncak objek.
- Galat kuantisasi maksimum antara titik asli dan titik wakilnya adalah diagonal voxel, yaitu $s_v\sqrt{3} \approx 0{,}087$ m.
- s_v yang lebih kecil memberi detail lebih halus tetapi titik lebih banyak; s_v yang lebih besar memberi peta lebih ringan tetapi detail objek mulai hilang.

8.3 Color Mapping Ketinggian

Setiap titik diwarnai berdasarkan koordinat z. Nilai z dinormalkan ke rentang $[0, 1]$:

$$\tilde{z} = \frac{z - z_{\min}}{z_{\max} - z_{\min}} \tag{18}$$

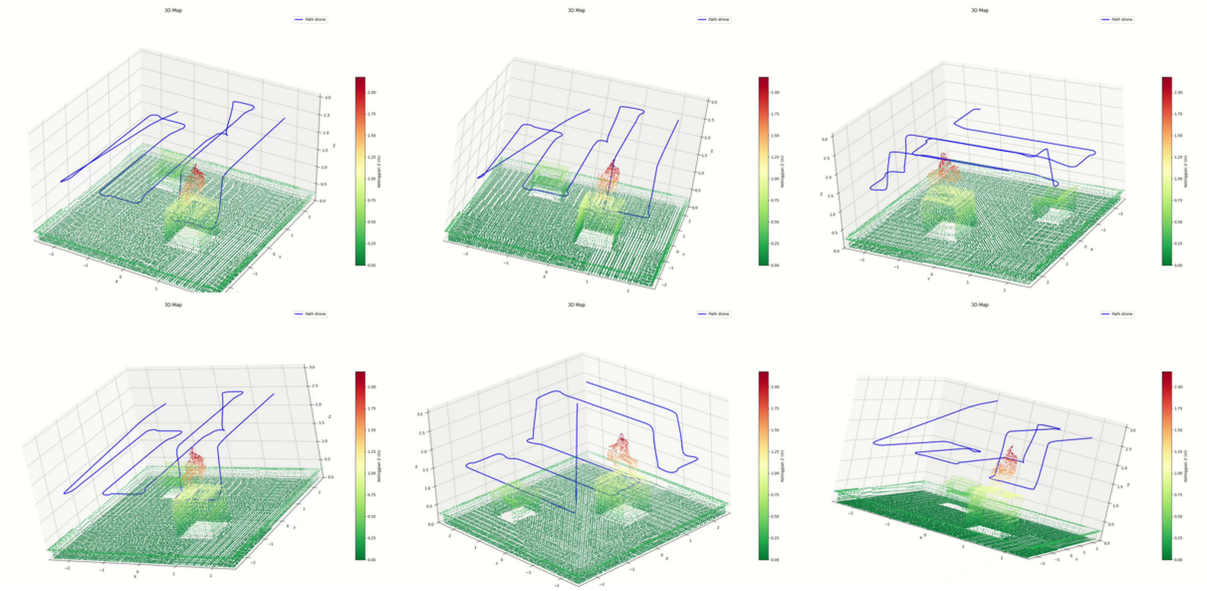
lalu dipetakan ke colormap `RdYlGn_r` (*Red–Yellow–Green* terbalik): $\tilde{z} = 0$ hijau (rendah), $\tilde{z} \approx 0{,}5$ kuning, dan $\tilde{z} = 1$ merah (tinggi). Colorbar ditambahkan sebagai legenda ketinggian dalam meter.

Agar proporsi peta tidak terdistorsi, batas sumbu diatur ke nilai minimum dan maksimum data $(\boldsymbol{lo}, \boldsymbol{hi})$ dan rasio kotak plot disamakan dengan dimensinya:

$$\text{box_aspect} = \boldsymbol{hi} - \boldsymbol{lo} = (\Delta x, \Delta y, \Delta z) \tag{19}$$

Jalur drone digambar di atasnya sebagai garis biru dari seluruh posisi yang dicatat.

9 Hasil dan Pembahasan



Gambar 4: Peta 3D dari enam sudut pandang. Warna titik menunjukkan ketinggian Z (hijau rendah, merah tinggi); garis biru adalah jalur drone.

Beberapa hal yang dapat diamati dari hasil:

1. Lantai muncul sebagai kisi titik hijau pada $z \approx 0$, tersebar rata berkat filter voxel.
2. Sofa tampak hijau-kuning (tinggi ≈ 1 m) dan ConcreteBlock beserta manekin berwarna kuning sampai merah (puncak ≈ 2 m), sesuai skala colorbar. Ketiga bagian objek (lantai, dinding samping, puncak) tertangkap berkat orientasi sensor.
3. Jalur biru memperlihatkan pola lawnmower yang naik di atas rintangan. Ini membuktikan perencanaan ketinggian adaptif berjalan dan drone tidak menabrak objek.
4. Area kosong di lantai tampak sebagai persegi putih di dekat objek. Ini adalah *occlusion shadow*, yaitu lantai di balik atau di bawah objek yang tidak terlihat sensor dari atas.

10 Kesimpulan

Sistem yang dibuat berhasil memetakan arena berisi rintangan secara 3D dengan drone. Rantai transformasi $\tilde{\boldsymbol{p}}_W = \boldsymbol{T}_W^B \boldsymbol{T}_B^S \tilde{\boldsymbol{p}}_S$ menempatkan setiap titik sensor ke frame global dengan benar, voxel filter menjaga ukuran peta tetap kecil, dan color mapping membuat informasi ketinggian mudah dibaca. Perencanaan ketinggian adaptif membuat drone dapat melewati objek yang lebih tinggi dari ketinggian terbang awal tanpa tabrakan.